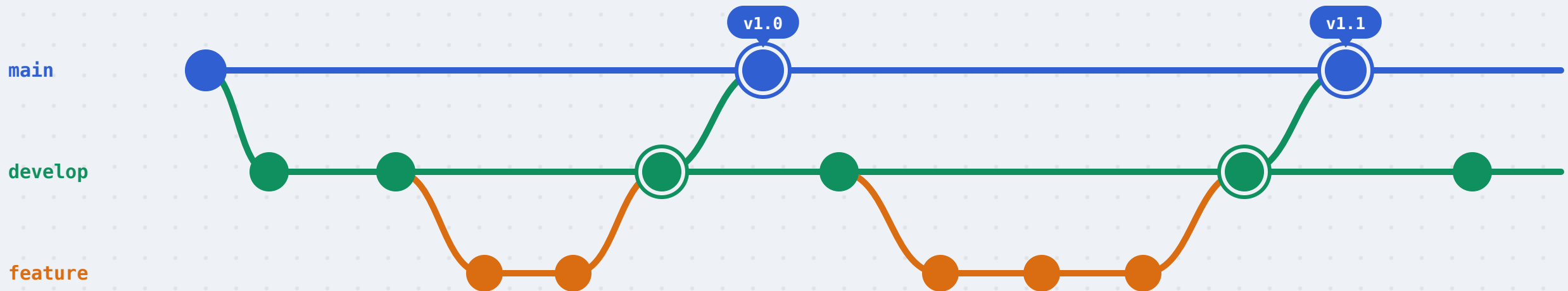


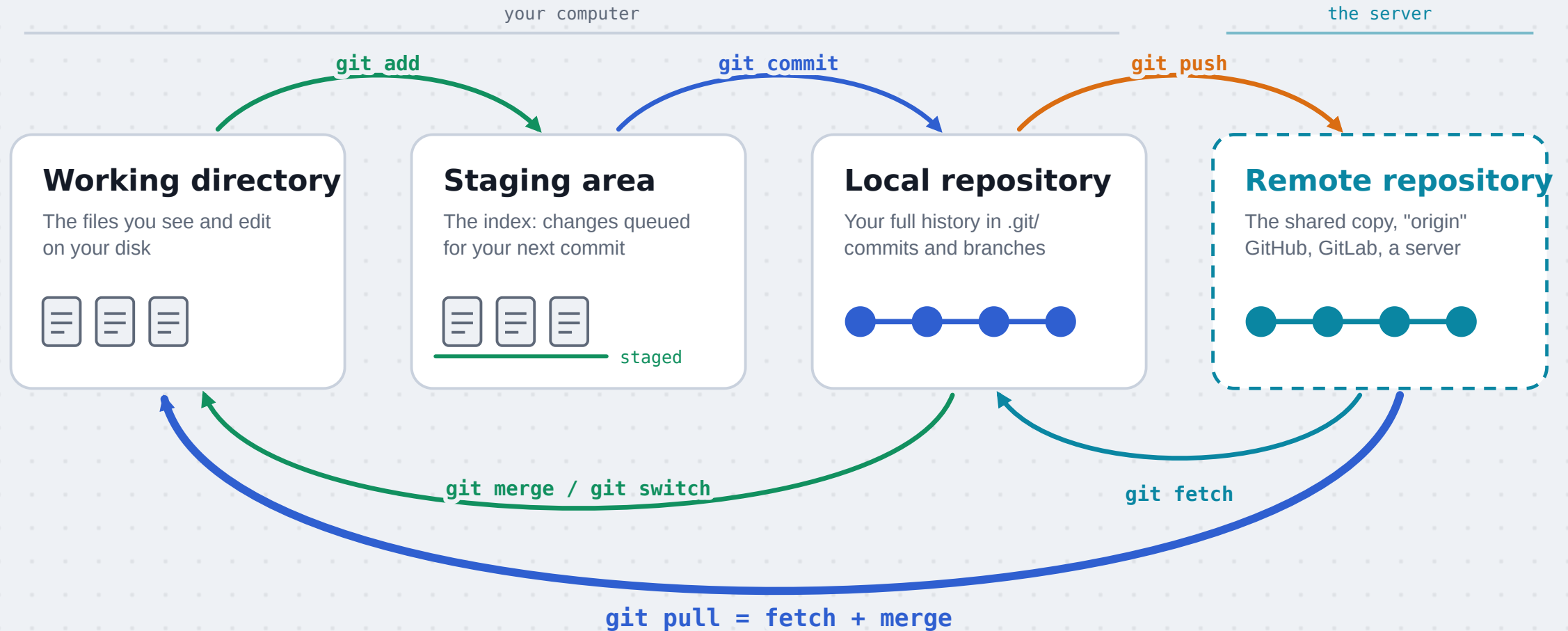
A VISUAL GUIDE

Git in Pictures

fetch, merge, pull, and the
branching strategies teams actually
use

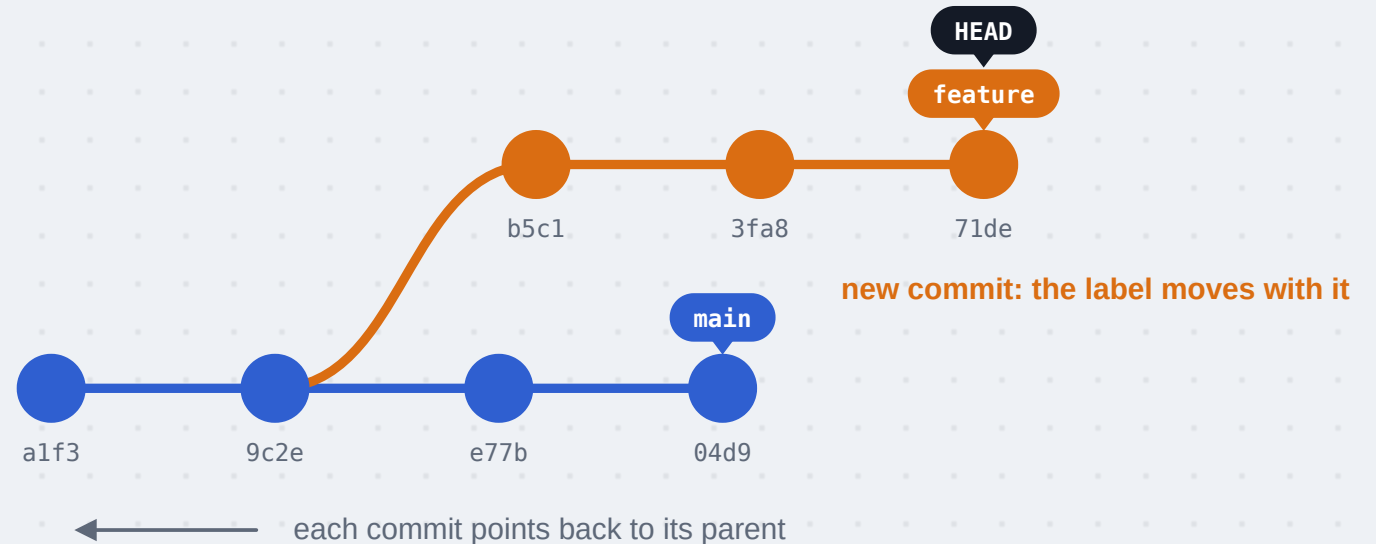


Your code lives in four places



History is a graph. Branches are labels.

- A **commit** is a full snapshot of your project, plus a pointer to its parent and a unique hash.
- A **branch** is a movable label that points at one commit. Creating one costs almost nothing.
- **HEAD** marks the branch you are on. Each new commit moves that label forward.



main vs origin/main

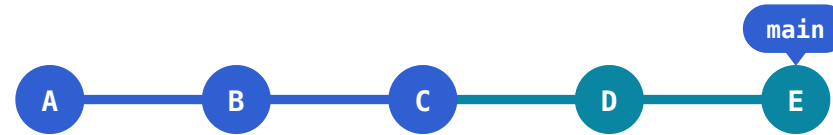
`main` is your branch. You move it.

`origin/main` is a read-only bookmark:
where `main` was on the server **the last time you talked to it**.

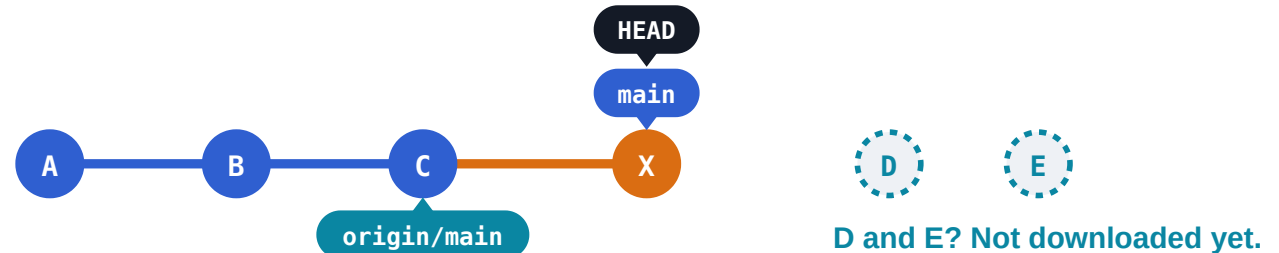
Teammates keep pushing. Your laptop does not know until you ask.

- shared history
- teammates' new commits
- your new commit

origin · the shared repository on GitHub



your laptop · local repository

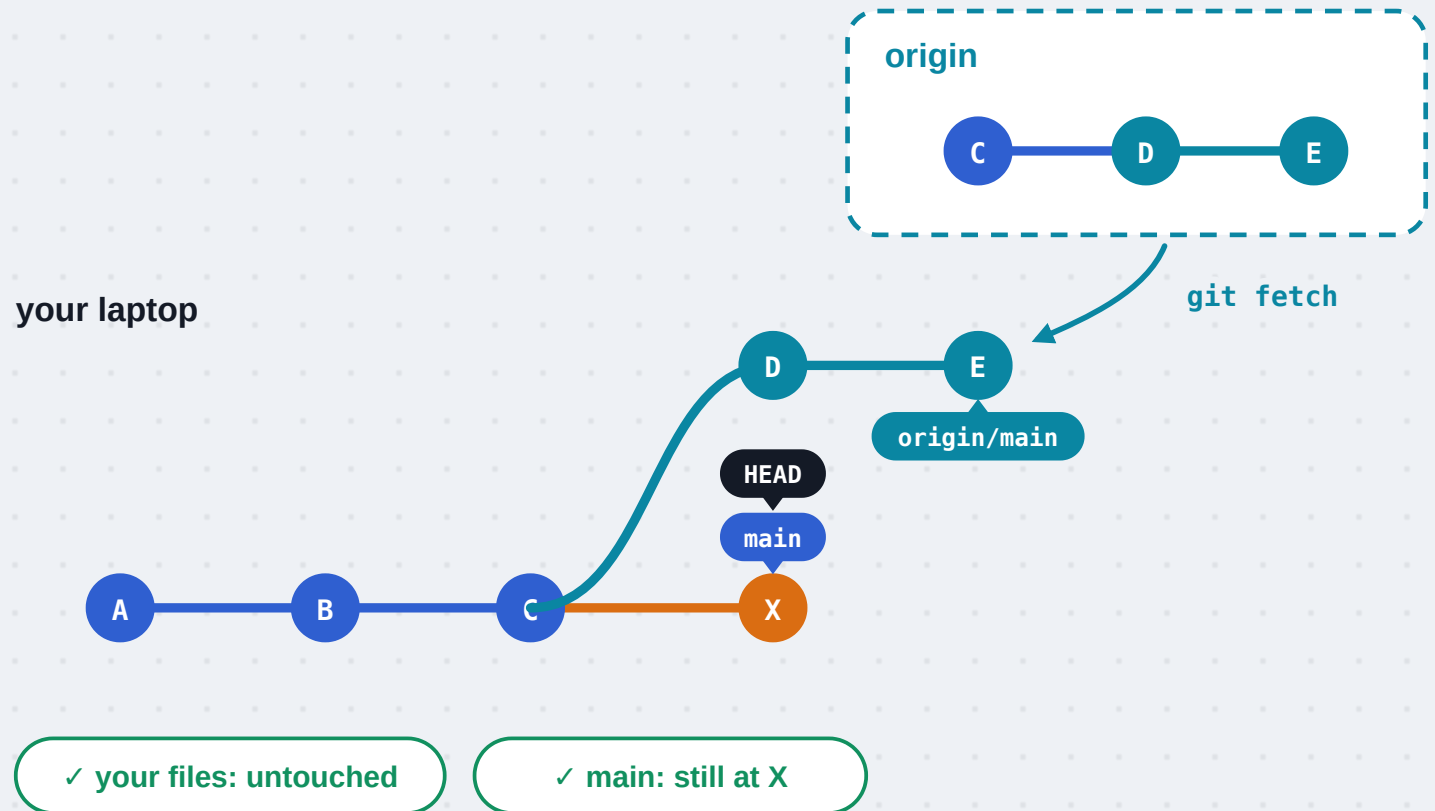


fetch downloads. It never touches your work.

```
$ git fetch origin
```

- Copies new commits from the server into your repository.
- Moves the `origin/*` bookmarks to match the server.
- Leaves `main`, your files and your staging area exactly as they were.

Safe to run any time. Then look before you leap: `git log main..origin/main`

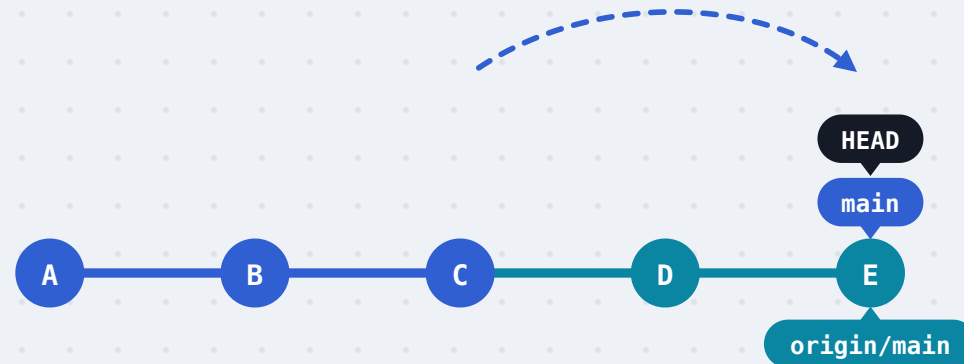


Fast-forward: the label just slides

```
$ git merge origin/main
```

If you have **no new commits** of your own, there is nothing to combine.

Git moves `main` forward to the same commit as `origin/main`. No merge commit, a perfectly straight history.



main slides forward. No new commit is created.

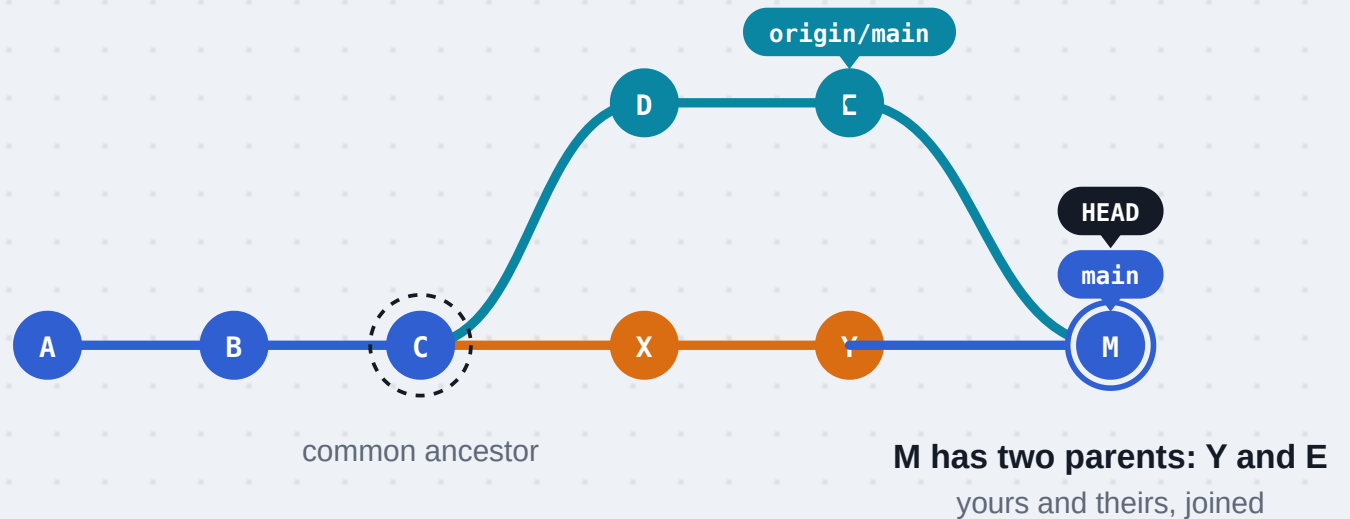
Diverged: a merge commit ties them together

```
$ git merge origin/main
```

You committed X and Y. Teammates pushed D and E. Both lines grew from C.

Git compares the two tips against their common ancestor C (a **three-way merge**) and records the result as a new commit **M**.

If both sides edited the same lines, Git stops and asks you: that is a conflict.



pull is two commands in one

git pull = **git fetch** + **git merge origin/main**

Downloads, then combines. Creates a merge commit if the histories diverged.

git pull --rebase = **git fetch** + **git rebase origin/main**

Downloads, then replays your local commits on top. Straight history, no merge commit.

git pull --ff-only = **git fetch** + **fast-forward, or stop**

Only updates if main can slide forward. If histories diverged, it stops and you choose.

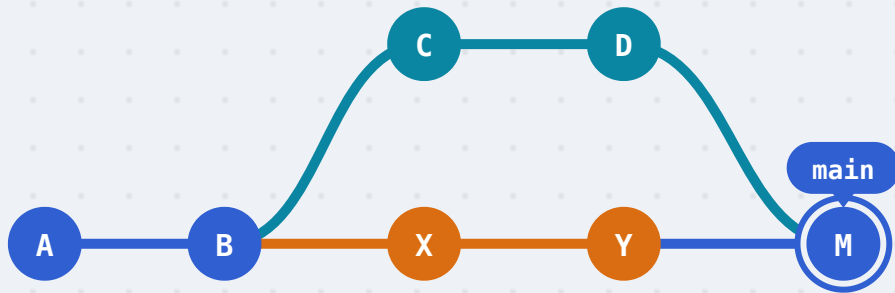
GOOD DEFAULT

Run `git config --global pull.ff only` once. Pull then never creates surprise merge commits, and you decide between merge and rebase when it matters.

TWO WAYS TO COMBINE

Merge keeps what happened. Rebase rewrites it straight.

`git merge origin/main`



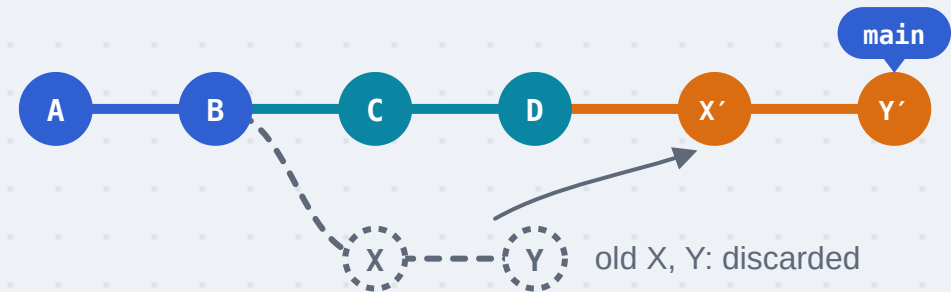
PROS

True history. Never rewrites commits.
Safe on shared branches.

CONS

Extra merge commits. The graph gets
busy on active branches.

`git rebase origin/main`



PROS

Linear, readable history. Easy to
bisect and review.

CONS

X and Y become new commits X' and
Y' with new hashes.

The rule: rebase only commits nobody else has. Never rebase a branch that others have already pulled.

Reading and resolving a conflict

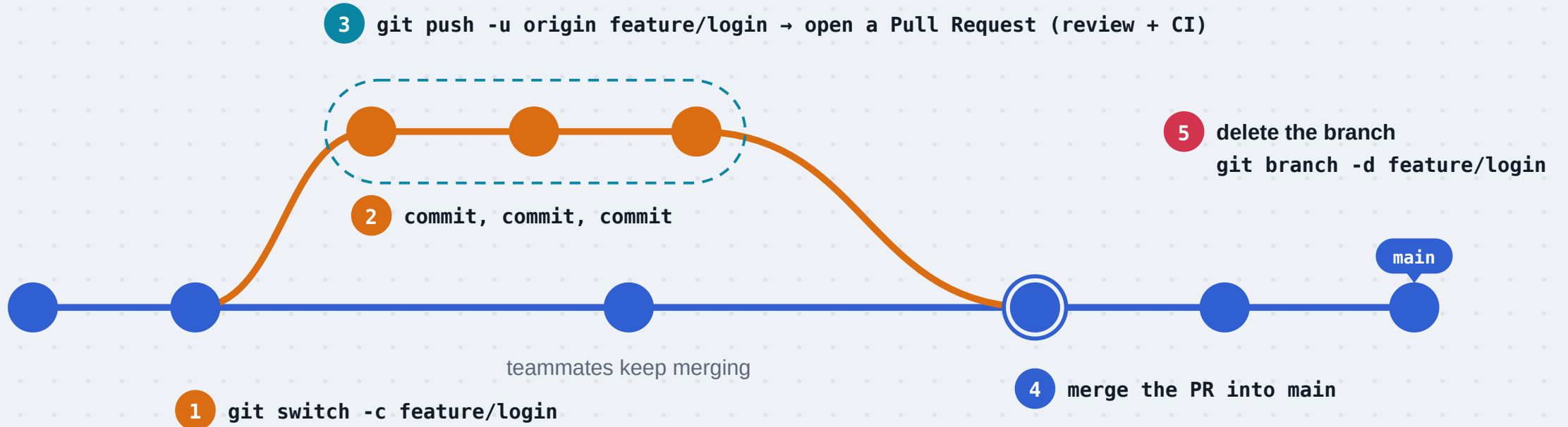
```
function connect() {  
  <<<<<< HEAD                                your side  
    const timeout = 30;  
  =====  
    const timeout = 60;  
  >>>>>> origin/main                          their side  
}
```

Git writes both versions into the file between markers. You decide what the final code is: yours, theirs, or a mix.

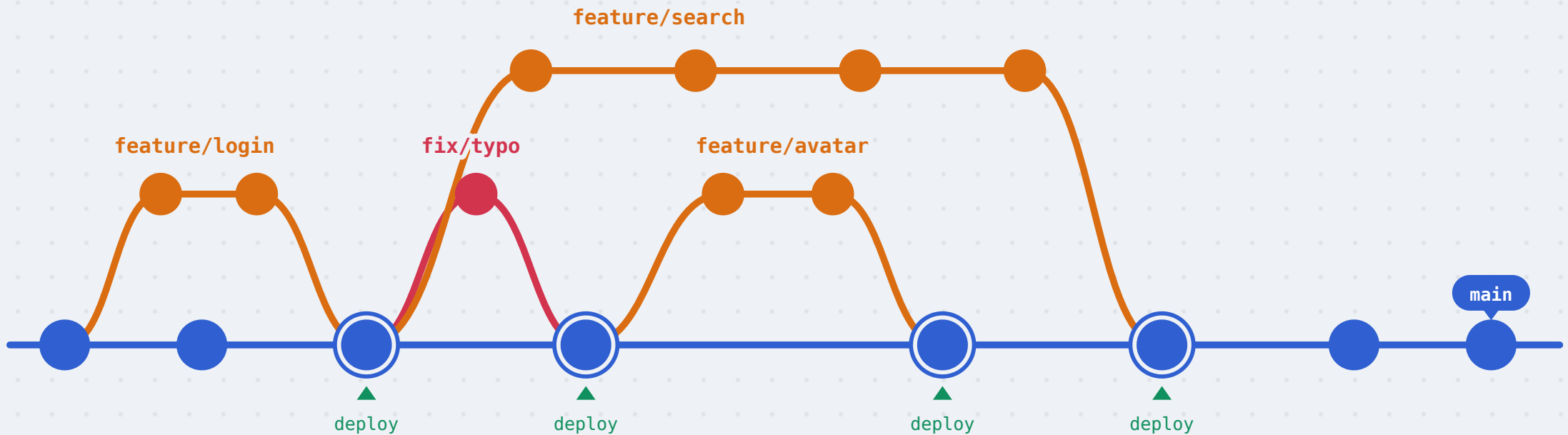
- 1 `git status` lists every file marked *both modified*.
- 2 Edit each file: keep the right code and delete all three marker lines.
- 3 `git add <file>` marks it resolved.
- 4 `git commit` to finish the merge, or `git rebase --continue` during a rebase.

Changed your mind? `git merge --abort` or `git rebase --abort` puts everything back as it was.

The life of a feature branch



One main branch, always deployable



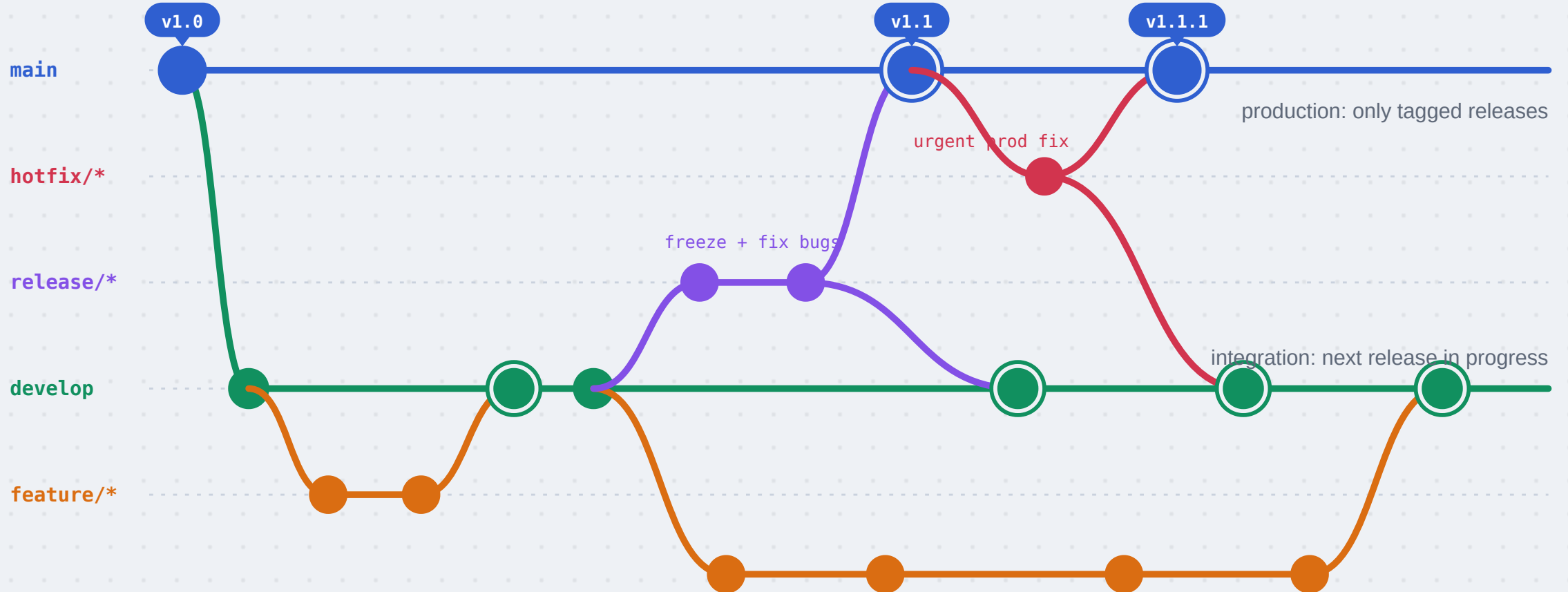
main is always releasable

one short branch per change

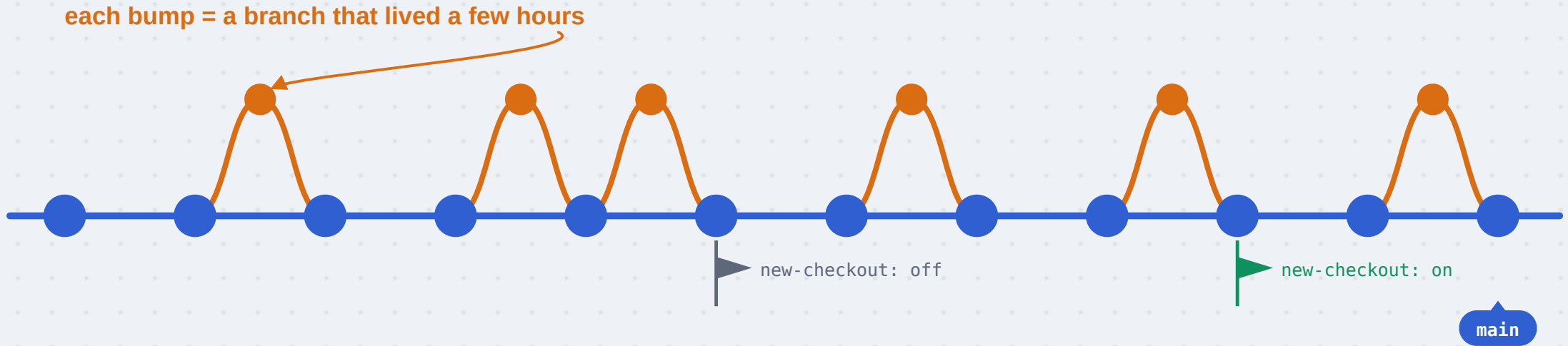
Pull Request + review + CI

merge, then deploy

Two long-lived branches, plus helpers



Everyone integrates into main, every day



branches live **hours**, not weeks

CI runs on every commit

unfinished work hides behind **feature flags**

release straight from main

CHOOSING

Which strategy fits your team?



GitHub Flow



Git Flow



Trunk-based

LONG-LIVED BRANCHES	main	main + develop	main only
BRANCH LIFETIME	days	days to weeks	hours
RELEASES	continuous deploy from main	versioned, scheduled (v1.1, v1.2)	continuous; flags control who sees what
NEEDS	good CI and code review	release discipline	strong tests, fast CI, feature flags
BEST FOR	web apps, small to mid teams	mobile apps, libraries, on-prem software with several versions in support	high-velocity teams, monorepos
MERGE PAIN	low	higher: long branches drift apart	lowest: changes stay small

A daily routine that avoids trouble

1 · START

Sync main

```
git switch main  
git pull --ff-only
```

2 · BRANCH

One branch per change

```
git switch -c feature/login
```

3 · WORK

Small commits

```
git add -p  
git commit -m "Add login form"
```

4 · STAY CURRENT

Catch up with main

```
git fetch  
git rebase origin/main
```

or `git merge origin/main` if the branch is shared

5 · SHARE

Push and open a PR

```
git push -u origin feature/login
```

6 · CLEAN UP

After the merge

```
git switch main && git pull  
git branch -d feature/login
```

Six rules worth keeping

Fetch often

It never changes your work, so run it whenever you want to see what's new.

Never rebase shared commits

Rewriting pushed history breaks everyone who built on it.

Commit before you pull

A clean working directory keeps every merge and rebase reversible.

Keep branches short

The longer a branch lives, the bigger and riskier its merge.

Pull with --ff-only

No surprise merge commits. You choose merge or rebase on purpose.

Protect main

Require a Pull Request, a review and green CI before anything lands.